

Design Optimisation

CSU34041

Yvette Graham



Today:

- Database Design
 - Design Guidelines – Central Guidelines (1-4)
 - Design Optimisation
 - Functional Dependency Definition
 - Quiz
-
- Normalisation
 - 1NF, 2NF, 3NF
 - Superkeys
 - Boyce-Codd Normal Form (BCNF)
 - Quiz

Database Design

Database Design

- In previous lectures we have discussed various aspects of database design
 - The Relational Model
 - The Entity Relationship Model
 - Entity Relationship Diagrams
 - Mapping to a Logical Database Design
 - Relational Database Schema
- We previously also looked at Functional Dependency as a way of modelling technique. We will now revisit Functional Dependency as a way of **assessing the quality of a relation design**

Database Design

- Need a formal method for analysing how relations and attributes are grouped
- A measure of appropriateness or quality, other than the intuition of the designer
 - To assess the quality of the design
- Measures
 - Design guidelines
 - Functional Dependencies
 - Normalisation

Database Guidelines

Design Guidelines

- A set of informal guidelines
 - Can be used as **measures** to determine the quality of a relation schema design
 - Measure 1: Attribute Semantics
 - Measure 2: Reduction of Redundancy
 - Measure 3: Reduction of NULLs
 - Measure 4: Generation of Spurious Tuples
- These measures are not always independent of one another

Attribute Semantics

- Attributes belonging to a relation have certain real-world meaning
- Semantics of a relation
 - refers to its **meaning** resulting from the interpretation of attribute values in a tuple
- Careful entity relationship modeling and accurate mapping to logical design help to ensure that a relational schema design has **clear meaning**

Guideline 1

Guideline 1

- Design a relation schema so that it is easy to explain its meaning
- Give relations and attributes meaningful names
- **Guideline 1:** Do not combine attributes from multiple entity types and relationship types into a single relation
 - Straightforward to interpret
 - Easy to explain its meaning

Violating Guideline 1

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------

- These relations violate guideline 1 by unnecessarily mixing attributes from distinct real-world entities
 - Employee and Department
 - Project and Employee

Reduction of Redundancy

- One goal of database schema design can be to minimise the storage space used
- Grouping attributes into relational schema has a significant effect on storage space by reducing storage of redundant information (e.g. information unnecessarily duplicated)

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn
-------	----------------	---------

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

Reduction of Redundancy

- Storing merged entities in single relations leads to another problem, *update anomalies*
- Update anomalies can be classified into:
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

Guideline 2

Guideline 2

- Design the relation schemas so that no insertion, deletion or modification anomalies are present
 - if anomalies are present, note them clearly and ensure all application programs operate correctly
- This second guideline is consistent with guideline 1

Insert Anomalies

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

- To insert a new employee into EMP_DEPT it is necessary to include either:
 - all attribute values for the department that the employee works for
 - NULLs, if the employee is not yet assigned
- Consistency becomes an issue
- Inserting a new department is difficult

Deletion Anomalies

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

- Deletion of Employees and Departments inextricably linked
 - If we delete the last employee currently assigned to a particular department, the information related to department is lost from the database
- This problem does not occur if using separate relations

Modification Anomalies

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------

- Modification makes consistency an issue
- If the manager of a department is changed
 - It is necessary to update the tuples of every employee who works for that department
 - Records can easily get out of sync
- This problem does not occur if using separate relations

Guideline 3

Guideline 3

- Avoid placing attributes in a relation schema whose values may frequently be NULL
 - if NULLs are unavoidable, ensure they apply in exceptional cases and not the majority of tuples
 - using space efficiently and avoiding NULL values are the main criteria for deciding upon attribute inclusion or exclusion
 - if excluded, create a separate relation for that attribute

Reduction of NULLs

- If many attributes do not apply to all the tuples of a relation, you end up with many NULL values in those tuples
 - Wastes storage space
 - Can make understanding attribute meanings more difficult
 - Leads to difficulty with Joins
 - Difficulty with aggregate functions
 - COUNT and SUM

Reduction of NULLs

- A NULL value may typically have two interpretations
 - missing but inapplicable
 - Zip Code for Irish Addresses (although we do now have EirCode!)
 - missing but applicable
 - an employee's date of birth is empty
 - unknown
 - known but absent

Violating Guideline 3

- If only 15% of employees have an office, then including an Office_Number attribute in the EMPLOYEE relation would violate guideline 3
 - Instead, create an EMPLOYEE_OFFICE relation
 - This could contain the attributes
 - Ssn, Office_Number
 - A tuple is entered in the relation for all employees with an office

Guideline 4

Generation of Spurious Tuples

- Joins across relations should only be performed on *Primary Key – Foreign Key* pairs of attributes
- If joins are performed on attributes which are not a Primary Key – Foreign Key pairing, spurious tuples are generated as a result
 - These tuples represent information which is not valid

Guideline 4

- Design relation schemas so that they can be joined using equality conditions on primary key, foreign key pairs
 - This guarantees that no spurious tuples are generated by the join
- Avoid relations that contain matching attributes that are not foreign key, primary key combinations

EMP_DEPT

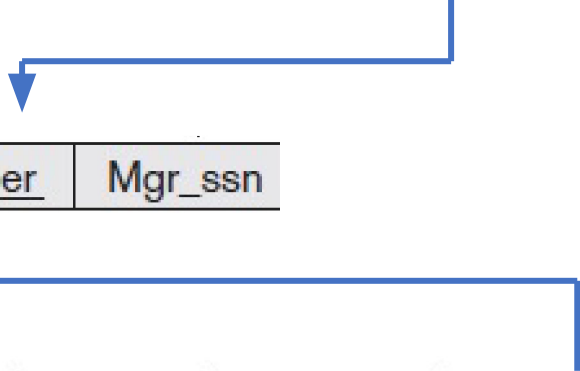
Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn
-------	----------------	---------

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------



Guideline 4

- Design relation schemas so that they can be joined using equality conditions on primary key, foreign key pairs
 - This guarantees that no spurious tuples are generated by the join
- Avoid relations that contain matching attributes that are not foreign key, primary key combinations

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn
-------	----------------	---------

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	------------------	---------------------

Design Guidelines Summary

- Informal measures used to determine the quality of a relational schema design
 - Ensure that attribute semantics are easily understood
 - Reduce the redundant information in tuples
 - Reduce the number of NULL values in tuples
 - Ensure that spurious tuples are not generated by enforcing primary key, foreign key matching

Design Optimisation

Using Functional Dependencies to optimise your relational model

- **Functional Dependency Analysis** is a Formal tool for analysing relational schemas
 - Enables the designer to detect and describe the problems in more precise terms
- One of the most important concepts in relational schema design theory
- Main tool for measuring the appropriateness of groupings of attributes into relations

Functional Dependency Definition

Functional Dependency: Definition

- A *functional dependency* is a constraint between two sets of attributes
- Suppose our relational database schema has n attributes
 - $A_1, A_2, \dots, A_n$
- Think of the whole database as being described by a single universal relation
 - $R = \{A_1, A_2, \dots, A_n\}$

Definition (cont)

- A functional dependency, denoted by $X \twoheadrightarrow Y$,
- Specifies a constraint on the possible tuples that can form a relation state r of R
 - between two sets of attributes X and Y
 - X and Y are subsets of the relation R
- The constraint is
 - for any two tuples t_1 and t_2 in $r(R)$ that have
$$t_1[X] = t_2[X]$$
 - they must also have $t_1[Y] = t_2[Y]$

Definition

- The values of the attribute set X from a tuple in r , uniquely determine the values of the attribute set Y

Or

- The values of the attribute set X from a tuple in r , *functionally* determine the values of the attribute set Y

— We can say that:

- There is a *functional dependency* from X to Y

or

- Y is *functionally dependent* on X

Definition

- The abbreviation for functional dependency is FD or f.d.
 - The set of attributes X is called the left-hand side of the FD, Y is called the right-hand side
- Thus
 - X functionally determines Y in a relation schema R if, and only if, **whenever two tuples agree on their X values, they must also agree on their Y values**

Things to Note

- If X is a candidate key of R , then
 - $X \twoheadrightarrow Y$ for any subset of attributes Y of R
 - Thus, $X \twoheadrightarrow R$
 - In other words, if X has to be unique for every instance of R , then X uniquely determines all the other attribute values of R
- If $X \twoheadrightarrow Y$ in R , this does not necessarily imply that $Y \twoheadrightarrow X$ in R
 - Not commutative

Identification of FDs

- A *functional dependency* is a property of the semantics or meaning of the attributes
 - A database designer will use their understanding of the semantics of the attributes of R to specify the functional dependencies that must hold on all instances of R
 - Entity relationship modeling supports the development of this understanding

Example

- Consider

EMP_PROJ					
<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation

- using the semantics of the attributes and relation, the following FDs should hold:

Ssn $\twoheadrightarrow$ Ename



Pnumber $\twoheadrightarrow$ {Pname, Plocation}



{Ssn, Pnumber} $\twoheadrightarrow$ Hours



Disproving a FD

- You cannot use a single set of data – $r(R)$ – to prove a FD, but you can use it to disprove a FD

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Bartram

- Can we prove? Text $\rightarrow$ Course

- Can we disprove? Teacher $\rightarrow$ Course

Disproving a FD

- You cannot use a single set of data – $r(R)$ – to prove a FD, but you can use it to disprove a FD

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Bartram

- Can't prove Text $\rightarrow$ Course

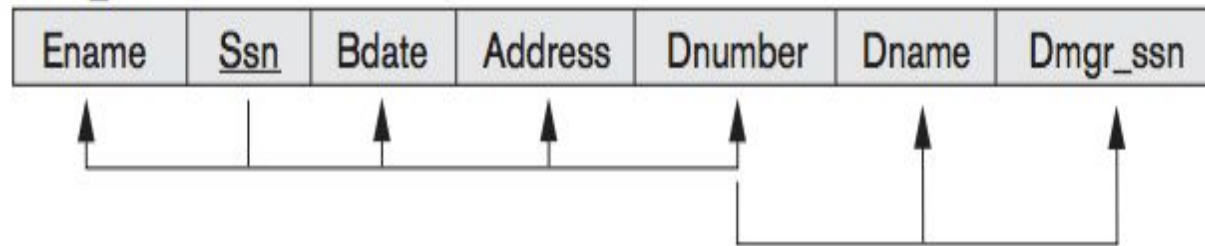
- Can disprove Teacher $\rightarrow$ Course

Constraints

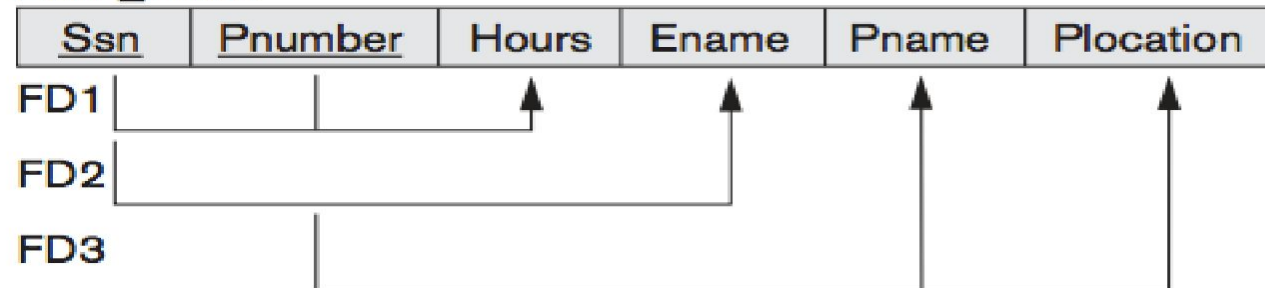
- Whenever the semantics of two sets of attributes of R indicate that a FD should hold, the dependency is specified as a *constraint*
- Hence, FDs are used to further enhance a relation schema R, by specifying constraints that must hold *at all times*

Diagrammatic Notation

EMP_DEPT



EMP_PROJ



Normalisation

Normalisation

- The normalisation process takes a relational schema through a series of tests to certify whether it satisfies a certain *normal form*
- There are a number of normal forms:
 - First Normal Form – 1NF
 - Second Normal Form – 2NF
 - Third Normal Form – 3NF
 - Boyce-Codd Normal Form – BCNF

Normalisation

- Evaluate each relation against the criteria for normal forms
 - Decompose relations where necessary
- Can be considered *relational design by analysis*
 - ER Modeling
 - Mapping to Relational Schema
 - Functional Dependencies
 - Normalisation

Normalisation

- Normalisation: process of analysing relation schemas based on their primary keys and functional dependencies to:
 - minimize redundancy
 - minimize insertion, deletion and modification anomalies
- Relations which do not pass the normal form tests are decomposed into smaller relation schemas

Normalisation

- Normalisation through decomposition must confirm two properties in the resulting database design
 - 1) Non-Additive or Lossless Join Property
 - This guarantees that **spurious tuple generation does not occur**
 - 2) Dependency Preservation Property
 - This ensures that **each functional dependency is represented in an individual relation**
- Provides database designers with:
 - a **formal framework** for analysing relations based on their **primary keys and functional dependencies**
 - a set of **normal form tests** that can be carried out on individual relation schemas so that the relational database can be normalised to the desired degree

First Normal Form (1NF)

First Normal Form

- In **1NF** all attribute values must be atomic
 - The word atom comes from the Latin atomis, meaning indivisible (or literally "not to cut")
- 1NF dictates that at every row-column intersection, there exists only one value, not a list of values
- The benefits from this rule should be fairly obvious.
 - If lists of values are stored in a single column, there is no simple way to manipulate those values.

1NF

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations



DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

How can we achieve 1NF?

Achieving 1NF

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

1NF

- 1NF also disallows multi-valued attributes that are themselves composite

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0

How can we achieve 1NF?

1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

How can we achieve 1NF?

Achieving 1NF

- Remove the attribute(s) that violates 1NF and put it into a separate, new relation
- Add the primary key of the original relation to the new relation
 - This will serve as a foreign key
- The primary key of the new relation is a composite primary key
- This *decomposes* a non-1NF relation into two 1NF relations

Achieving 1NF

- Follow the same steps as with a single multi-valued attribute

EMP_PROJ

Ssn	Ename	Pnumber	Hours
-----	-------	---------	-------

EMP

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

1NF Example

- A database stores information about people
 - People can own more than one car
 - People can have more than one phone number

Person

<u>PPS_num</u>	Name	Car_Reg_No	Car_Model	Car_CC	Phone_Num
----------------	------	------------	-----------	--------	-----------

Is this table in 1NF and if not what would be the normalization of this table so that it is in 1NF?

1NF Example

- A database stores information about people
 - People can own more than one car
 - People can have more than one phone number

Person

<u>PPS_num</u>	Name	Car_Reg_No	Car_Model	Car_CC	Phone_Num
----------------	------	------------	-----------	--------	-----------

<u>PPS_num</u>	Name
----------------	------

<u>PPS_num</u>	<u>Phone_Num</u>
----------------	------------------

<u>PPS_num</u>	Car_Reg_No	Car_Model	Car_CC
----------------	------------	-----------	--------

Second Normal Form (2NF)

Full Functional Dependency

- A FD $X \twoheadrightarrow Y$ is said to be *fully functional dependent* if the removal of any single attribute from the set of attributes X means that the dependency no longer holds
 - Think of X as a composite primary key
 - All the other attributes must be dependent upon the full key, not just part of it
- for any attribute $A \in X$
 $(X - \{A\})$ does not functionally determine Y

Partial Functional Dependency

- A FD $X \twoheadrightarrow Y$ is said to be a *partial functional dependency* if a single attribute can be removed from the set of attributes X yet the dependency still holds
- for any attribute $A \in X$

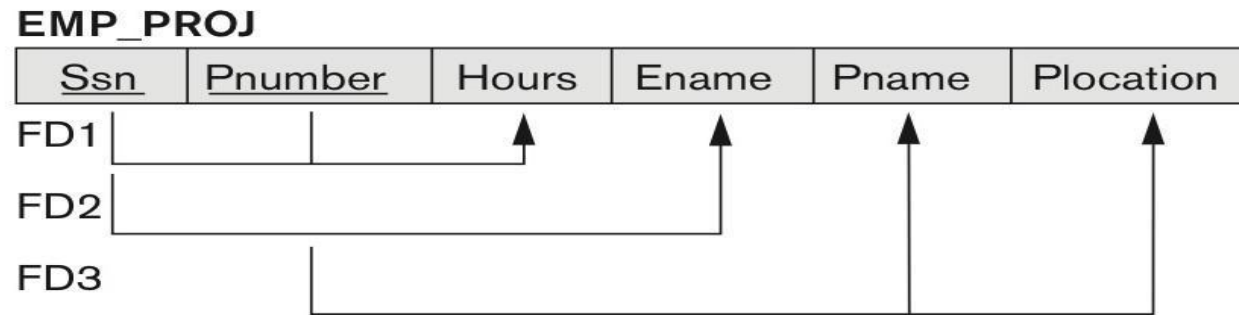
$$(X - \{A\}) \twoheadrightarrow Y$$

Second Normal Form (2NF)

- A table is said to be in Second Normal Form (2NF) if:
 - it is 1NF compliant
 - every non-key column is *fully functionally dependent* on the entire primary key.
- In other words:
 - tables should only store data relating to one "thing" (or entity)
 - that entity should be described by its primary key.

2NF Example

Supposed we had a single table EMP_PROJ with primary key Ssn & Pnumber and attributes of Hours, Ename, Pname and Plocation



Such a table, with SSn and Pnumber as a composite Primary key would mean that each attribute should Functionally Depend on this Composite Primary Key

- Composite Primary Key – {Ssn, Pnumber}
- {Ssn,Pnumber} → Hours
- {Ssn,Pnumber} → Ename
- {Ssn,Pnumber} → {Pname, Plocation}

2NF Example (cont)

- Composite Primary Key – {Ssn, Pnumber}
- {Ssn,Pnumber} $\twoheadrightarrow$ Hours – *Full FD*
- {Ssn,Pnumber} $\twoheadrightarrow$ Ename – *Partial FD*
- {Ssn,Pnumber} $\twoheadrightarrow$ {Pname, Plocation} – *Partial FD*

Note 1:

Ename only partially depends on the Composite Primary key as Ename does not functionally depend on the Pnumber part of the Primary key

Note 2:

Similarly Pname and Plocation only partially functionally depend on composite primary key as Pname and Plocation only functional depend on the Pnumber part of the Primary key (i.e. they do not depend at all on the SSn part of the composite Primary Key)

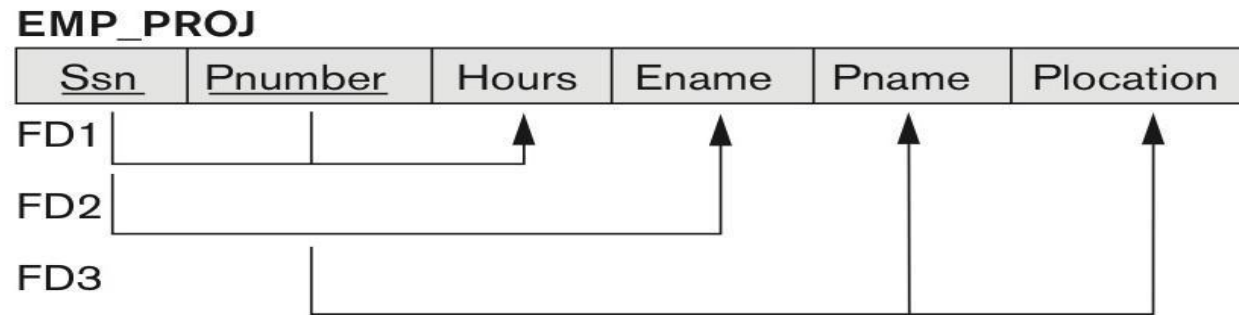
Achieving 2NF

o achieve 2NF do the following:

1. Limit the Functional Dependencies of attributes in the relation to *only* the parts of the primary key that they are dependent upon
 2. Decompose the relation into separate relations using these FDs
 3. The primary key of the new relation is the left-hand side of the FD
- This *decomposes* a non-2NF relation into a set of new relations which are in 2NF

2NF Example

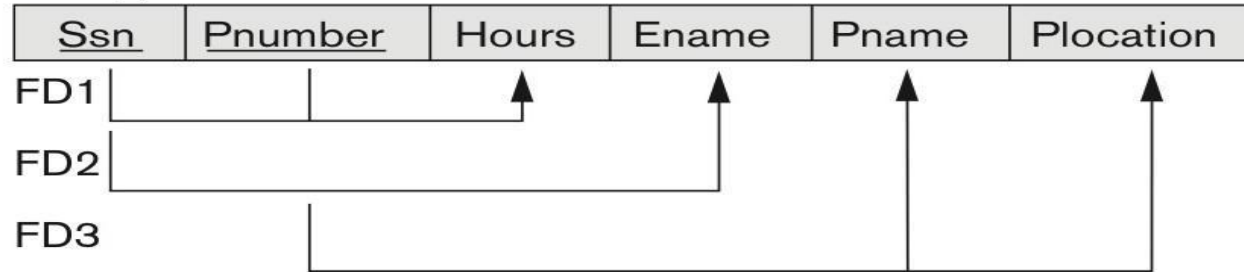
Taking our example again:



1. Limit the Functional Dependencies of the attributes in the relation to only the parts of the primary key that they are dependent upon
2. Decompose the relation into separate relations using these functional dependencies
3. The primary key of the new relations is the left-hand side of the functional dependencies

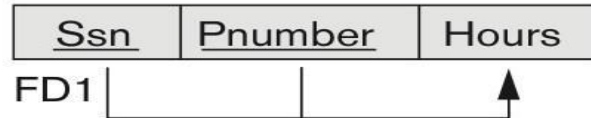
2NF (cont)

EMP_PROJ

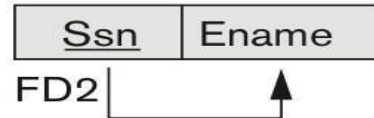


Is normalized to:

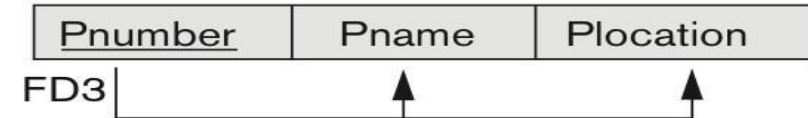
EP1



EP2



EP3

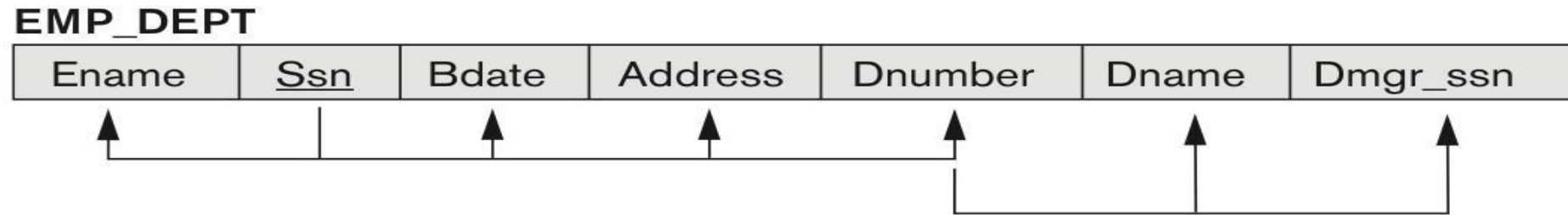


Third Normal Form (3NF)

Third Normal Form

- A table is said to be in Third Normal Form (3NF) if:
 - it is 2NF compliant
 - No non-key attributes are *transitively dependent* upon the primary key.
- A functional dependency $X \twoheadrightarrow Y$ in the relation R, is said to be *transitive dependent* if:
 - there exists a set of attributes Z in R which is neither a candidate key nor a subset of any key of R
 - and both $X \twoheadrightarrow Z$ and $Z \twoheadrightarrow Y$ hold true

Transitive Dependency

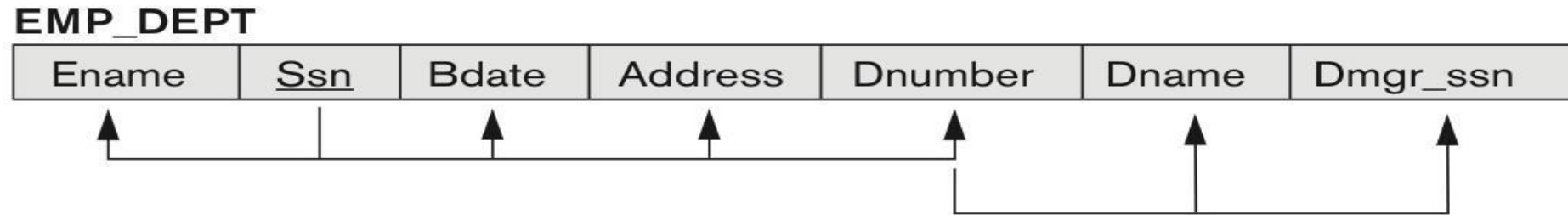


- Ssn $\twoheadrightarrow$ Dmgr_ssn is a transitive dependency through Dnumber
 - Ssn $\twoheadrightarrow$ Dnumber and Dnumber $\twoheadrightarrow$ Dmgr_ssn hold
 - Dnumber is not a key itself or a subset of any key of EMP_DEPT

Achieving 3NF

- **Identify** any transitive dependencies in the relation
- **Decompose** the relation into two separate relations using these transitive dependencies
- The primary key of the new relation is the **middle attribute** of the transitive dependency
- This *decomposes* a non-3NF relation into two new 3NF relations

3NF Example



- To normalise to 3NF:
 1. **Identify** any transitive dependencies in the relation
 2. **Decompose** the relation into two separate relations using these transitive dependencies
 3. The primary key of the new relation is the **middle** attribute of the transitive dependency

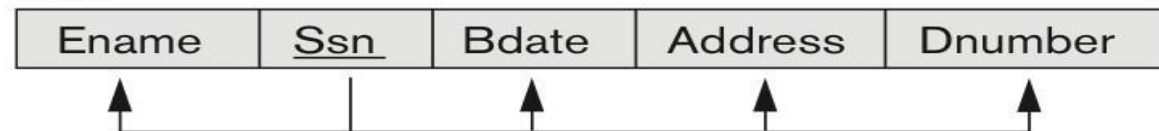
3NF (cont)

EMP_DEPT

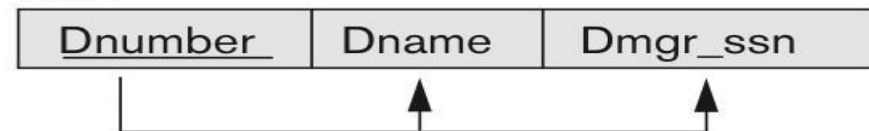


Is normalised to:

ED1



ED2



2NF and 3NF

- Any functional dependency in which the left hand side is...
 - a non-key attribute or
 - a component attribute of a composite primary key...is a problematic FD
- 2NF and 3NF remove these problem Functional Dependency by decomposing them into new relations

Summary 1NF, 2NF & 3NF

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

Superkeys

Superkey

- A *superkey* SK is any set of attributes in the relation R, whose combined values will be unique for every tuple
 - $t_1[SK] \neq t_2[SK]$
- Every relation has one default superkey – the set of all its attributes
 - as, by definition, every instance of a relation (i.e. a tuple in that relation) must be unique

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
- Primary Key
- Superkeys

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
- Primary Key
- Superkeys

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
- Superkeys

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
 - Engine_No
- Superkeys

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
 - Engine_No
- Superkeys
 - {Engine_No, Reg_Year, Reg County, Reg Num}

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
 - Engine_No
- Superkeys
 - {Engine_No, Reg_Year, Reg County, Reg Num}
 - {Engine_No, Reg_Year, Reg County, Reg Num, Model}

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
 - Engine_No
- Superkeys
 - {Engine_No, Reg_Year, Reg County, Reg Num}
 - {Engine_No, Reg_Year, Reg County, Reg Num, Model}
 - {Reg_Year, Reg County, Reg Num}

Superkey

Car

<u>Engine_No</u>	Reg_Year	Reg_County	Reg_Num	Model	CC
------------------	----------	------------	---------	-------	----

- Candidate Keys
 - Engine_No
 - {Reg_Year, Reg County, Reg Num}
- Primary Key
 - Engine_No
- Superkeys
 - {Engine_No, Reg_Year, Reg County, Reg Num}
 - {Engine_No, Reg_Year, Reg County, Reg Num, Model}
 - {Reg_Year, Reg County, Reg Num}
 - {Reg_Year, Reg County, Reg Num, Model}

Boyce-Codd Normal Form

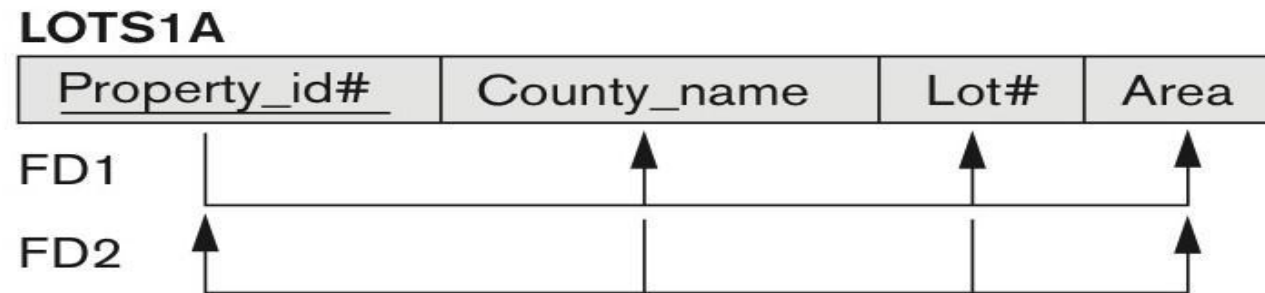
Boyce-Codd Normal Form

- BCNF was created to be a simpler form of 3NF
 - Sometimes called 3.5NF
- However, it was found to be stricter than 3NF
 - Every relation in BCNF is also in 3NF
 - Every relation in 3NF is not necessarily in BCNF
- A table is said to be in Boyce-Codd Normal Form (BCNF) if:
 - whenever a functional dependency $X \twoheadrightarrow Y$ holds in the relation R,

X is a superkey of R

BCNF

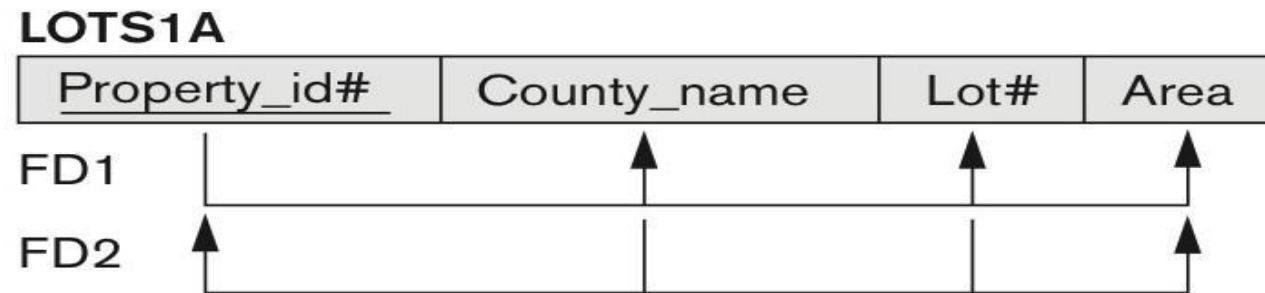
- Consider the following relation schema which describes parcels of land available for sale in different counties



- Two candidate keys
 - Property_id#
 - {County_name, Lot#}

BCNF

- Consider the following relation schema which describes parcels of land available for sale in different counties



- Two candidate keys
 - Property_id#
 - {County_name, Lot#}

Property_id#	County	Lot#	Area
65	Dublin	701	Fingal
66	Cork	702	Bantry

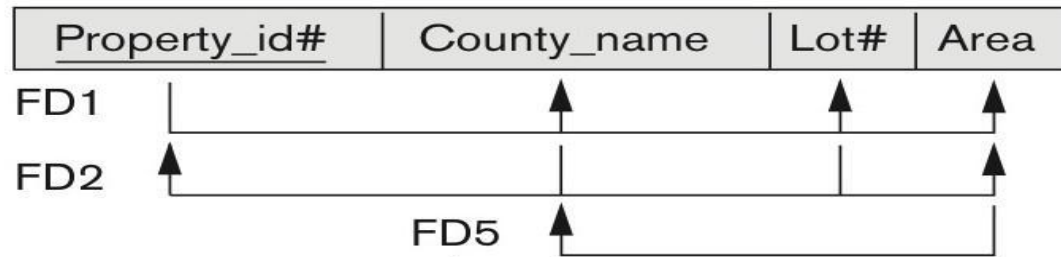
BCNF

- Suppose that there are thousands of lots in the LOTS relation, but:
 - the lots are from only two counties: Wicklow and Monaghan
 - Valid lot sizes in Wicklow are only 0.5, 0.6, 0.7, 0.8, 0.9, and 1.0 acres
 - Valid lot sizes in Monaghan are 1.1, 1.2, ..., 1.9, and 2.0 acres.

BCNF

- In such a situation there would be an additional functional dependency
 - $\text{Area} \twoheadrightarrow \text{County_name}$

LOTS1A



- This breaks BCNF as Area is not a superkey

Achieving BCNF

- Identify all functional dependencies in the relations
 - Identify any FDs where the left-hand side is not a superkey
- Decompose the relation into separate relations, creating a new relation for the offending FD
- The primary key of the new relation is the left hand attribute of the offending functional dependency
- This *decomposes* a non-BCNF relation into two new BCNF relations

BCNF Example

LOTS1A

<u>Property_id#</u>	County_name	Lot#	Area
---------------------	-------------	------	------



BCNF Normalization

LOTS1AX

<u>Property_id#</u>	Area	Lot#
---------------------	------	------

LOTS1AY

<u>Area</u>	County_name
-------------	-------------

Normalisation

- Normalisation tests a relation schema to certify whether it satisfies a *normal form*
 - First Normal Form – 1NF
 - Second Normal Form – 2NF
 - Third Normal Form – 3NF
 - Boyce-Codd Normal Form – BCNF¹
- Evaluate each relation against the criteria for each normal form in turn
 - Decompose relations where necessary

¹ Boyce/Codd rule for determining redundancy is rule

"Every determinant must be a candidate identifier"

to put it another way:

"all attributes in a relation should be dependent on the key, the whole key and nothing but the key"

Steps for Modelling prior to creating a Database

Steps for Modelling

- I. Identify and model the information as an ER Model
 - Represent the required entities and attributes
 - Represent the relationships between those entities and any attributes of those relationships
 - Define the cardinality of the relationships
 - Define the participation constraints of the relationships (total or partial)
- II. Map from the ER model to a relational schema
- III. Identify the functional dependencies in the relation schema
- IV. Normalise the relation schema to BCNF

Denormalization

- In practice, there are times when fully normalized tables can slow down performance too much
- This leads to step taken to denormalize the tables
- The correct design process is as follows:
 1. Design database to create fully normalized tables (correctness)
 2. Denormalize tables only where needed to speed up performance

Denormalization needs to be done with extreme caution as it can introduce anomalies that we would like to avoid.

Summary:

- Database Design
 - Design Guidelines – Central Guidelines (1-4)
 - Design Optimisation
 - Functional Dependency Definition
 - Break
-
- Normalisation
 - 1NF, 2NF, 3NF
 - Superkeys
 - Boyce-Codd Normal Form (BCNF)
 - Quiz